Metro Ticketing System using ServiceNow - Capstone Project Documentation

1. Introduction

Project Title: Metro Ticketing System using ServiceNow

2. Project Overview

Purpose: To deploy a cloud-native transit pass fulfillment and dynamic fare calculation engine on ServiceNow, featuring encrypted QR gate validation.

Features: Interactive Service Portal interface, distance-based pricing logic, automated RITM order processing, encrypted QR code attachment, automated email alerts, and administrative executive dashboards.

The ServiceNow-Powered Commuter Transit Platform is an enterprise solution designed to modernize public transportation by completely digitizing ticket procurement, tariff computation, and passenger validation. By replacing physical ticket booths with a fast self-service portal, the system eliminates queue bottlenecks at station gates and speeds up commuter onboarding.

Built within the ServiceNow Service Catalog framework, the interface allows commuters to define travel parameters such as origin station, arrival station, departure date, and passenger count with automated field validation. Once a route is chosen, Catalog Client Scripts trigger asynchronous GlideAjax calls to custom Script Includes, retrieving distance matrix data and calculating exact ticket costs instantly without manual intervention.

Upon checkout, the platform automatically generates a primary Request (REQ) and an underlying Requested Item (RITM) record to drive workflow fulfillment. Integrated server scripts encode crucial transaction details including RITM reference number, route endpoints, travel date, and passenger count into an encrypted QR pass image, which is instantly attached to the record and made available on the portal.

For transit management, the application provides live reporting dashboards that display daily booking volumes, high-density travel corridors, peak operational hours, and total revenue metrics. Simultaneously, the system's notification engine dispatches automated email confirmations containing full booking details and digital QR pass links directly to commuters. Ultimately, this project demonstrates how ServiceNow can be extended beyond conventional ITSM to build scalable, custom transportation automation systems.

3. Architecture

Frontend: ServiceNow Service Portal / Custom Widget UI Pages
Backend: ServiceNow Flow Designer, Business Rules, Script Includes, Catalog Client Scripts, GlideAjax
Database: ServiceNow Custom Data Tables(Custom Ticket Table / RITM) storing route matrices, tariff schedules, passenger details, and pass transactions

1. Presentation Layer (User Experience & Interface)

The presentation tier is deployed using ServiceNow Service Portal components and custom Catalog Widget interfaces, serving as the primary touchpoint for commuter ticket procurement. Passengers interact with dynamic, responsive order forms to capture travel inputs including origin stop, destination node, schedule slot, and passenger count. Client Scripts run actively on the browser side to execute immediate field validations, enforce future-date selection constraints, and trigger real-time, dynamic fare previews without causing page refreshes.

2. Application Logic Layer (Backend Processing)

The application layer governs all server-side business rules, pricing logic, and task orchestration. It operates using:

Business Rules: Server-side scripts executed automatically upon record creation or modification. They manage automated ticket ID assignments, validate fare integrity, and initiate downstream QR pass attachments once an order is submitted.
Script Includes & GlideAjax: Reusable, object-oriented JavaScript modules that process complex server tasks asynchronously, such as querying multi-station distance matrices, evaluating tariff rules, and encoding encrypted pass parameters.
Flow Designer Automation: Orchestrates the end-to-end lifecycle of the transaction, coordinating state transitions from initial request submission to pass approval, QR encoding, and automated confirmation dispatch.

3. Data Layer (Database)

All system records and transactional data are structured across optimized ServiceNow Data Tables, including:

Custom Data Tables (Custom Ticket Table / RITM **) & Requested Item Table:**
Store master station indices, fare matrix parameters, and active ticket records.

Transaction Datastores: Maintain structured properties for each booking, including unique REQ/RITM identifiers, route endpoints, total calculated fare, passenger counts, departure timestamps, and linked QR pass image payloads.

4. Integration & Output Layer

Following successful order validation, backend utility scripts compile key pass metadata into an encrypted payload to generate a single-use QR code. This image is automatically linked as a file attachment to the RITM record. Concurrently, the ServiceNow Notification Engine captures the event trigger and dispatches an automated email containing full ticket details and an inline pass link to the commuter.

5. Reporting & Monitoring Layer

System administrators leverage ServiceNow Reporting and Executive Dashboards to gain real-time visibility into transit network activity.

Daily and monthly ticket issuance volume
Total gross revenue generated
High-density routes and top origin-destination pairs
Peak passenger booking hours and transit trend analysis

4. Setup Instructions

Prerequisites: ServiceNow Developer Instance
Installation: Configure service catalog items, create ticket table, configure business rules and client scripts.

Step 1: Create Service Catalog Item

- Navigate to Service Catalog → Maintain Items.
- Click New and define the catalog item as Metro Ticket Booking
- Assign to the appropriate Catalog Source Station (Reference or Dropdown)
    - ➢ Destination Station (Reference or Dropdown)
    - ➢ Journey Date (Date Picker)
    - ➢ Number of Passengers (Number Field)

Step 2: Create Backend Data Structure

- Navigate to System Definition → Tables
- Create a custom table (e.g., u_metro_ticket) OR use RITM table (sc_req_item) if extending catalog flow
- Add required fields:
    - ➢ Ticket Reference Number (u_ticket_number)
    - ➢ Origin Station (u_source)
    - ➢ Destination Station(u_destination)
    - ➢ Fare Amount
    - ➢ Passenger Count
    - ➢ Encrypted QR Attachment Link
    - ➢ QR Code (Image)

Step 3: Configure Client Scripts

- Navigate to Service Catalog → Catalog Client Scripts.
- Create an onChange script on u_destination_station to perform immediate validation:
    - ➢ Enforce non-identical station selection (Source ≠ Destination).
    - ➢ Invoke GlideAjax asynchronously to calculate and render a real-time fare preview before submission.
    - ➢ Create an onSubmit script to enforce mandatory fields and validate travel date limits.

Step 4: Configure Business Rules

- Navigate to System Definition → Business Rules.
- Create an after-insert Business Rule on the ticket/RITM table.

Define automated execution logic:

- ➢ Validate final transaction parameters against backend pricing rules
- ➢ Ticket number generation
- ➢ Execute server-side calls to initiate QR pass encoding.

Step 5: Implement Script Includes

- Navigate to System Definition → Script Includes.
- Create a client-callable JavaScript module (MetroTicketingUtil):
    - **calculateFare(source, destination, count):** Queries u_metro_fare to compute exact distance-based total fare.
    - **generatePassQR(ritmSysId):** Compiles transaction metadata, generates encrypted pass payload, and writes the image attachment directly to the target record.

Step 6: Configure Email Notifications

- ➢ Navigate to System Notification → Email → Notifications.
- ➢ Create a new notification record triggered when a ticket/RITM is set to Approved or Work in Progress.
- ➢ Set recipients to the Requested For user.
- ➢ Build the HTML notification template containing booking summary details, travel guidelines, and the inline embedded QR ticket pass.

Step 7: Build Reporting Dashboards

- ➢ Navigate to Reports → Create New.
- ➢ Construct data visualization widgets:
  - **Total Ticket Sales:** Single-score widget tracking daily pass volumes.
  - **Revenue Analytics:** Time-series line chart tracking daily and weekly fare collection.
  - **Popular Corridors:** Bar chart showing top origin-destination pairs.
- ➢ Embed report widgets onto the Service Portal landing page for real-time administrative visibility.

Step 8: End-to-End System Testing

- ➢ Access the Service Portal as a standard commuter user.
- ➢ Submit a ticket request selecting origin, destination, schedule date, and passenger count.
- ➢ Verify full fulfillment workflow:
  - Successful creation of REQ and RITM records.
  - Accurate dynamic fare calculation.
  - Automated attach-and-render of the generated QR code.
  - Instant receipt of email confirmation with pass attachments.
  - Live updates across operational dashboard reports.

Final Note
Once all components are configure

5. Folder / Component Structure

1. Service Catalog Layer

- ➢ Contains the Metro Ticket Booking Catalog Item.
- ➢ Defines user-facing portal variables: Source Station, Destination Station, Journey Date, and Passenger Count.
- ➢ Drives commuter interaction and self-service booking flows on the Service Portal.

2. Client-Side Logic

- ➢ Built using Catalog Client Scripts and UI Policies.
- ➢ Performs real-time field validations (e.g., preventing identical source/destination selection).
- ➢ Manages dynamic field behavior, visibility rules, and asynchronous GlideAjax fare previews.

3. Server-Side Business Logic

- Built using **Business Rules** and **Script Includes** (MetroTicketingUtil).
- Executes distance-based fare calculations, ticket number formatting, and backend QR pass generation.
- Guarantees consistent workflow execution upon record creation and state changes.

4. Data Management Layer

- Utilizes custom schema tables (u_metro_station, u_metro_fare) alongside transaction records (sc_req_item / u_metro_ticket).
- Persists core travel metadata, passenger counts, total tariffs, lifecycle states, and attached QR payloads.

5. Reporting & Analytics Layer

- Built using ServiceNow Reports and executive Dashboards.
- Tracks total ticket sales, revenue trends, high-density routes, and peak travel hours.
- Delivers real-time operational insights for transit administrators.

Key Structural Principle

- **UI Layer** → Service Catalog + Service Portal
- **Logic Layer** → Client Scripts + Business Rules + Script Includes
- **Data Layer** → ServiceNow Data Tables (u_metro_station, u_metro_fare, sc_req_item)
- **Insight Layer** → Reports & Interactive Dashboards

6. Running the Application

Access Service Portal URL Submit
ticket via catalog item
Backend processes run automatically via Business Rules

Step 1: Access the Service Portal

Users open the ServiceNow Service Portal via the instance URL and navigate to the Metro Ticket Booking catalog item under the transportation services catalog section.

Step 2: Fill Ticket Booking Form

The commuter inputs necessary journey parameters:

- Source Station
- Destination Station
- Journey Date
- Number of Passengers

Client Scripts and UI Policies automatically enforce input accuracy:

- Mandatory fields are populated.
- Origin and destination stations are not identical.
- Valid date ranges and formats are selected prior to submission.

Step 3: Submit Ticket Request

Submitting the form creates an active service transaction:

- A record is generated in the sc_req_item table (or custom ticket table).
- The system automatically assigns a unique tracking identifier.

Step 4: Backend Processing
Upon record insertion, backend workflows execute automatically:

- ➢ Business Rules run on insert to evaluate fare parameters against station matrices.
- ➢ Reusable Script Includes calculate total tariffs and write metadata to the record store.

Step 5: QR Code Generation
The backend encoding utility compiles essential ticket parameters including Ticket ID, route endpoints, schedule date, and passenger count into an encrypted, single-use QR pass attachment linked directly to the ticket record.

Step 6: Notification Delivery
The ServiceNow Notification Engine fires upon request approval:

- ➢ An automated email confirmation is dispatched to the commuter's registered address.
- ➢ The user receives booking confirmation along with ticket details and QR code

Step 7: Monitoring and Verification
Commuters and transit administrators monitor system operations through:

- ➢ Live ticket status tracking within the Service Portal and RITM record view.
- ➢ Real-time executive reports and analytics dashboards displaying daily booking volumes and revenue metrics.

7. API / Automation Logic

The ServiceNow Metro Ticket Generating System primarily utilizes ServiceNow's native server-side automation framework to handle fare calculations, record updates, and workflow progression within the platform. For QR code generation, an external web API service is integrated to produce machine-readable pass images.

1. Internal Automation (ServiceNow Platform)
The following core operations are handled natively within ServiceNow:

- ➢ Executing after-insert Business Rules on record creation.
- ➢ Calculating distance-based tariffs dynamically via reusable Script Includes.
- ➢ Auto-generating unique tracking identifiers (MTK-2026-XXXXX).
- ➢ Managing request lifecycle states (REQ / RITM).
- ➢ Triggering automated email notifications upon successful booking.

These operations ensure that the system remains fully automated without manual intervention.

2. External QR Code Generation API
An external QR API is integrated within a server-side Script Include (MetroTicketingUtil) to generate single-use travel passes.
Working Flow:

- ➢ **Payload Assembly:** Upon ticket creation, the system compiles key metadata (Ticket Number, Source, Destination, Journey Date, and Passenger Count) into a structured string payload
- ➢ **REST API Call:** A REST message call passes the encoded payload to the external QR generation endpoint.
- ➢ **Response Processing:** The API returns a binary image stream or base64 encoded string representing the generated QR code.
- ➢ **Record Attachment:** The returned image is attached directly to the target RITM/ticket record and embedded into the commuter's confirmation email.

3. Automation Trigger Point

- **Trigger Event:** Executed via an after-insert Business Rule on the ticket table.
- **Execution Guardrail:** Triggers only after target record creation is committed to the database, ensuring zero orphaned API calls for unsubmitted transactions.

4. Data Flow Summary

ServiceNow Ticket Record → Business Rule Trigger → Script Include Processing → QR API Call → QR Image Returned → Stored in Ticket Record → Notification Sent

8. Authentication

The ServiceNow Metro Ticket Generating System leverages ServiceNow's native security infrastructure and Role-Based Access Control (RBAC) framework to enforce secure data handling across all application components.

1. Authentication Mechanism

User authentication is managed by the ServiceNow platform itself. Users must log in using valid ServiceNow credentials before accessing the Service Portal or backend modules. Session management, login validation, and user identity verification are handled by the platform's security layer.

2. Role-Based Access Control (RBAC)

System privileges and module visibility are regulated through defined security roles:
Typical roles include:

- **End-User / Commuter Role (**snc_external / u_metro_user**):** Accesses the Service Portal interface; submits new ticket requests and views self-created RITM records.
- **Transit Support Representative** (u_metro_support**):** Views and manages all active ticket records, monitors order states, and addresses commuter fulfillment inquiries.
- **System Administrator (**admin**):** Maintains full platform access, including table schemas, Script Includes, Business Rules, Flow Designer actions, and system properties

3. Data Security Controls

Data privacy is enforced across multiple operational layers:

- **Table-Level Access Control Lists (ACLs):** Restricts direct read, write, and create operations on u_metro_station, u_metro_fare, and ticket tables based on assigned roles.
- **Field-Level Restrictions:** Hides backend processing attributes (such as internal sys_id references and API transaction payloads) from end-user portal views.
- **Record-Level Security Rules:** Enforces row-level ACLs ensuring standard commuters can only view ticket records where Requested For matches their active user ID (javascript:gs.getUserID()).
- **Server-Side Enforcement:** Prevents frontend logic tampering by executing all critical validation rules and fare calculations exclusively within server-side scripts.

4. Portal Security

- Access to the Metro Ticket Booking catalog widget requires an authenticated user session;

unauthenticated attempts automatically redirect to the instance login page.

- ➢ All submitted payload parameters undergo server-side validation upon record transaction to block unauthorized parameter overrides.

5. QR Code Access Control

Generated QR codes are tied to authenticated ticket records, ensuring:

- ➢ QR pass images are generated strictly post-insertion for verified, paid bookings.
- ➢ Once attached to the RITM record, pass metadata fields are set to read-only state to prevent post-generation tampering

9. User Interface

The ServiceNow Metro Ticket Generating System provides an intuitive, responsive interface built using the ServiceNow Service Portal, optimized to streamline pass procurement and deliver real-time operational visual analytics

1. Ticket Booking Form (Service Portal)

The primary interface is a structured Service Catalog form where users input journey details. Key features include:

- ➢ **Clean & Responsive Layout:** Designed for frictionless commuter interaction across desktop and mobile browsers.
- ➢ **Input Parameters:** Dedicated fields for Source Station, Destination Station, Journey Date, and Passenger Count.
- ➢ **Dynamic Validation Engine:** Client Scripts and UI Policies perform instant checks to enforce non-matching origin/destination selections and future-date constraints.
- ➢ **Asynchronous Fare Preview:** Calls GlideAjax in real time to calculate and render the estimated tariff prior to order submission.

2. Ticket Confirmation Page

Upon successful request checkout, commuters are directed to an interactive order summary view displaying:

- ➢ **Tracking Identifier:** Unique ticket pass number (MTK-2026-XXXXX).
- ➢ **Journey Breakdown:** Confirmed origin station, destination station, scheduled travel date, and passenger count.
- ➢ **Billed Tariff:** Final calculated total fare.
- ➢ **Digital Pass Attachment:** Embedded high-resolution QR code pass ready for gate turnstile scanning.

The QR code is prominently displayed and serves as the digital ticket for metro entry verification.

3. Design Principles

- ➢ **Frictionless Workflow:** Minimal input steps required to complete ticket procurement.
- ➢ **Visual Consistency:** Standardized styling aligned with ServiceNow Service Portal UI guidelines.

- ➢ **Cross-Device Responsiveness:** Fully adaptable layout across mobile, tablet, and desktop interfaces.
- ➢ **Inline System Feedback:** Dynamic field validation prompts and status notifications prevent user error prior to submission.

10. Testing

The ServiceNow Metro Ticket Generating System underwent comprehensive end-to-end verification through a structured test strategy to validate order intake workflows, pricing calculation accuracy, server script execution, and digital pass generation.

1. Test Scope

Testing coverage spanned across all core application layers:

- ➢ Service Catalog UI & Commuter Booking Form
- ➢ Dynamic Fare & Tariff Computation Engine
- ➢ Business Rule Execution & Workflow Triggers
- ➢ External API Integration & QR Code Generation
- ➢ Email Notification Engine & Pass Attachment

2. Test Scenarios and Cases

Test Case 1: End-to-End Booking Request Creation

- ➢ **Test Input:** Valid Source Station, Destination Station, Journey Date, and Passenger Count
- ➢ **Expected Result:** Order processes successfully, generating valid REQ and RITM records in sc_req_item.
- ➢ **Execution Status:** PASS

Test Case 2: Origin & Destination Validation Constraints

- ➢ **Test Input:** Identical origin and arrival stations selected (e.g., Source = Destination).
- ➢ **Expected Result:** Client Script blocks form submission and displays an inline error message.
- ➢ **Execution Status:** PASS

Test Case 3: Automated Fare Calculation Verification

- ➢ **Test Input:** Selected station pair with known distance tariff parameters.
- ➢ **Expected Result:** Script Include accurately computes total fare based on route distance and passenger volume.
- ➢ **Execution Status:** PASS

Test Case 4: Unique Ticket Tracking ID Generation

- ➢ **Test Input:** New ticket request submission.
- ➢ **Expected Result:** System automatically assigns a unique tracking identifier format
- ➢ **Execution Status:** PASS

Test Case 5: Encrypted QR Pass Generation & Attachment

- ➢ **Test Input:** Valid ticket record inserted into the target data store.
- ➢ **Expected Result:** After-insert Business Rule executes REST API call to generate and attach the QR pass image.
- ➢ **Execution Status:** PASS

Test Case 6: Email Notification Dispatch & Pass Delivery

- ➢ **Test Input:** Ticket status updated to active/confirmed.
- ➢ **Expected Result:** Notification Engine sends an automated email containing journey summary details and the embedded QR pass.
- ➢ **Execution Status:** PASS

11. Known Issues

Despite successful implementation of the Metro Ticket Booking System, certain limitations exist due to platform constraints and external dependencies.

1. External QR API Dependency

Digital travel pass generation relies on an external REST service endpoint. In the event of external API downtime, network latency, or service throttling, QR code generation and pass attachment to RITM records may experience delays or failures.

2. Absence of Offline Gate Verification

Validation requires continuous network connectivity to the ServiceNow platform instance for live ticket state verification. Offline QR pass scanning and entry validation at station gate turnstiles are not supported in the current release.

3. Static Tariff Structure

Fare calculations rely on fixed, predefined station-pair matrices (u_metro_fare). Dynamic pricing logic based on peak travel hours, real-time station congestion, or demand-based surcharges is not currently implemented.

4. Lack of Integrated Payment Gateway

The application automates order creation, tariff calculation, and pass issuance but lacks live payment gateway integration (e.g., Stripe, PayPal, or UPI). Booking submissions complete automatically without processing live financial transactions.

5. Basic Pass Security & Cryptography

Generated QR passes encode standard transaction metadata (Ticket Number, Route, Date, Passenger Count) but do not incorporate high-grade cryptographic signatures or dynamic rotation mechanics, presenting potential exposure to pass duplication if unmonitored.

6. High-Concurrency Scalability Scope

System testing was conducted under standard development instance load conditions. High-concurrency performance metrics such as processing thousands of simultaneous ticket requests during peak transit hours have not been stress-tested.

12. Future Enhancements

The ServiceNow Metro Ticket Generating System can be further extended to enhance platform capabilities, increase transactional security, and achieve full production readiness for enterprise transit deployment.

1. Mobile QR Validation & Gate Scanner Integration

Develop a dedicated, native mobile scanner application or ServiceNow Agent Mobile integration for station gate attendants and automated turnstile hardware. This will enable real-time, high-speed QR pass scanning and instantaneous platform state verification to accelerate passenger throughput at entry gates.

2. Integrated Payment Gateway Processing

Integrate secure, enterprise-grade payment gateways (e.g., UPI, Stripe, credit/debit card processors, or digital wallets) via REST integration points. This will establish an end-to-end purchasing workflow, automating payment validation prior to triggering fare confirmation and QR pass generation.